K-Means Clustering

K-Means is an unsupervised learning algorithm used to group similar data points into clusters.

Goal

Divide data into K groups (clusters) where:

- Points in the same cluster are similar
- Points in different clusters are different

How K-Means Words(Steps)

1. Choose the no of clusters K
2. randomly initialize K centroids
3. Assign each data point to the nearest centroid
4. Recalculate the centroid(mean of cluster points)
5. Repeat steps 3 and 4 until centroid stop changing ##bcs new val add centrod change

Cluster evaluation

1. Inertia(Within-cluster sum of squares)

- Measures how tightly data points are grouped
- Sum of squared distances btw points and their centroid
- Lower inertia = better clustering

2. Elbow Method

Used to find the best value of K.

- Plot:K vs Inertia
- Look for the elbow point where inertia stops decreasing sharply

That value of K is considered the best choice.

3. Silhouette Score

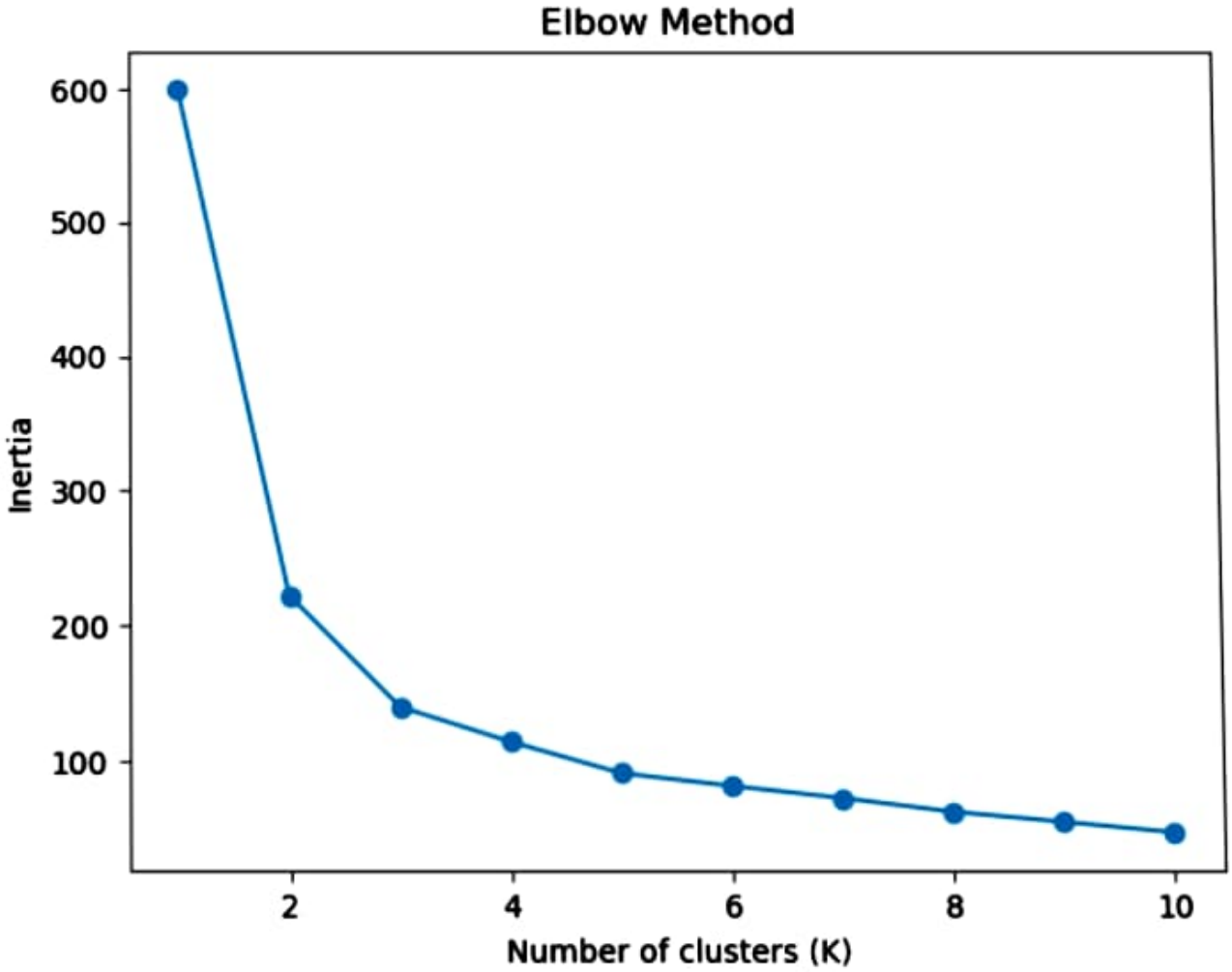
Measures how well clusters are separated.

Range: -1 to +1

- +1 -> Perfect clustering
- 0 -> Overlapping clusters
- -1 -> Wrong clustering

1]:

```
import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# 1. Load dataset
iris = load_iris()
X = iris.data

# 2. Feature scaling (important for K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Elbow Method (Inertia)
inertia = []
K_range = range(1, 11)
```

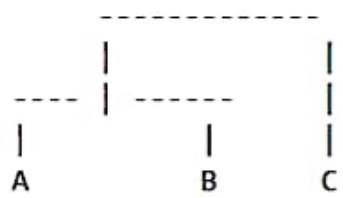


Silhouette Score: 0.45994823920518635

```python
# 3. Elbow Method (Inertia)
inertia = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.plot(K_range, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.show()

# 4. Fit final KMeans (choose K = 3 for iris)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# 5. Silhouette Score
score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)

# 6. Visualize clusters (first 2 features)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            s=300, c='red', marker='X')

plt.title("K-Means Clustering Result")
plt.show()
```


K-Means Clustering Result
3
2
1
0
−1
−2
−2
−1
0
1
2

Example



- A and B merge first
- Then C joins
- Height indicates similarity

Linkage methods

It determines how the distance btw clusters is calculated

1. Single linkage

Uses the minimum distance btw clusters.

Distance = Minimum distance btw any 2 points

Pros

- Finds irregular shaped clusters

Cons

- Sensitive to noise

2. Complete Linkage

Uses the maximum distance btw clusters.

Hierarchical Clustering

It is an unsupervised learning algorithm that groups similar data points into clusters by building a hierarchy of clusters.

Unlike K-Means,you do not need to specify the no of clusters(K) initially.

How hierarchical clustering works

There are 2 approaches:

1. Agglomerative(Bottom-Up)

- Starr with each data point as its own cluster
- Merge the 2 closest clusters
- Repeat until all points belong to 1 clusters

2. Divisive(Top-Down)

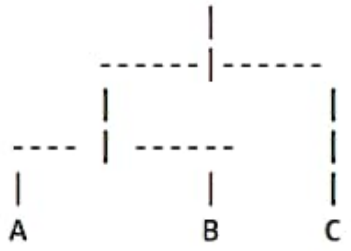
- Start with all data points in 1 cluster
- Split clusters recursively
- Continue until each point forms its own cluster

Dendrogram

A dendrogram is a tree like diagram that shows how clusters are merged.

- Each leaf represents a data point
- Branches show clusters merging
- Height represents distance btw clusters

2. Count how many vertical branches it crosses



If a horizantal line crosses 3 branches:

```
No of clusters = 3
```

Best practice

Choose the cut where the vertical distance is largest.

[2]:
```python
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

# Load dataset

iris = load_iris()
X = iris.data

# Scale features

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Distance = Maximum distance btw any 2 points

Pros

- Creates compact clusters

Cons

- Sensitive to outliers

3. Average Linkage

Uses the average distance btw all points.

Distance = Average pairwise distance

Pros

- Balanced clustering

4. Ward Linkage(Most popular)

Merges clusters that minimize variance.

Pros

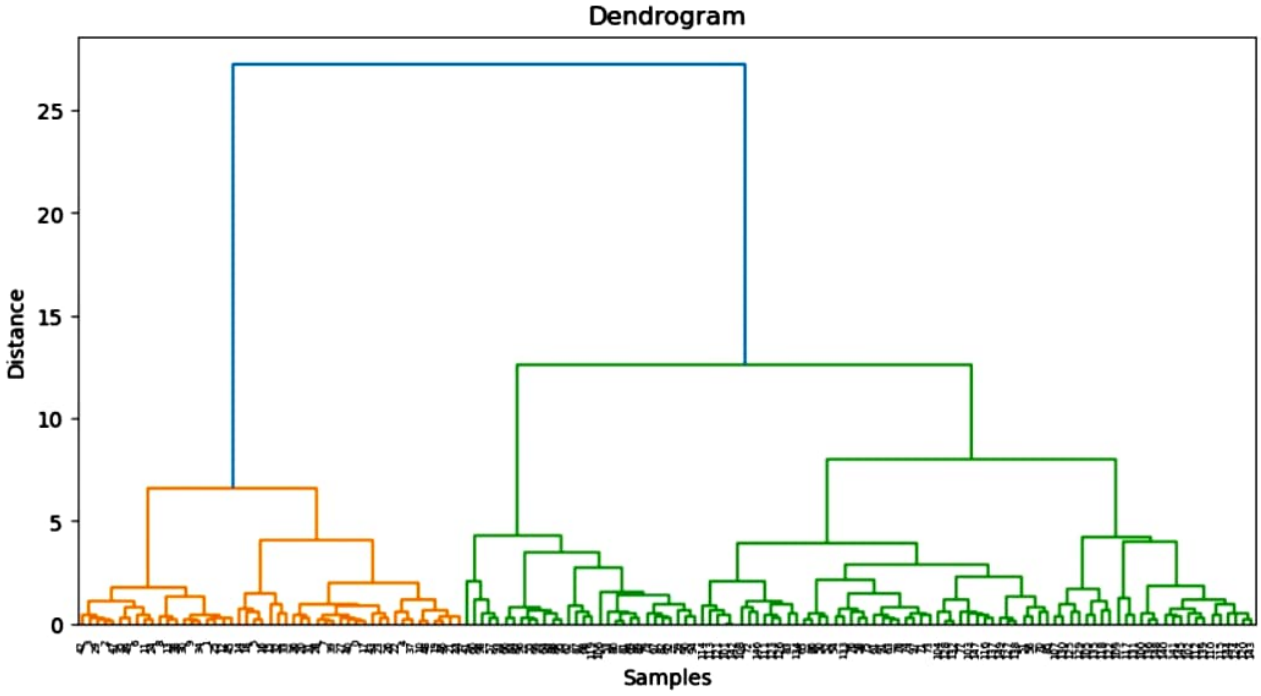
- Produces well balanced clusters
- Often gives the best results

Use the Dendrogram

Rule

1. Draw a horizantal line

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Create linkage matrix

linked = linkage(X_scaled, method='ward')

# Plot dendrogram

plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()

# Hierarchical Clustering(bottom-up)

hc = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward'
)

labels = hc.fit_predict(X_scaled)
print("Cluster Labels:")
print(labels)
```




```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 2 1 1 1 1 1 1 1 0 0 0 2 0 2 0 2 0 2 2 0 2 0 2 0 2 2 2 2 0 0 0 0
 0 0 0 0 0 2 2 2 2 0 2 0 0 2 2 2 2 0 2 2 2 2 0 2 2 0 0 0 0 0 0 2 0 0 0 0
 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```